NAME : P SAI VENKAT ROLL NO : 2403A510G0

SUBJECT : AI ASSISTED CODING TEST : LAB TEST –2

Lab Set 5

Question 1: AI-Assisted Code Review

Task 1:

Submit a sample Python script (with inefficient logic and poor naming) to an AI model for
review.

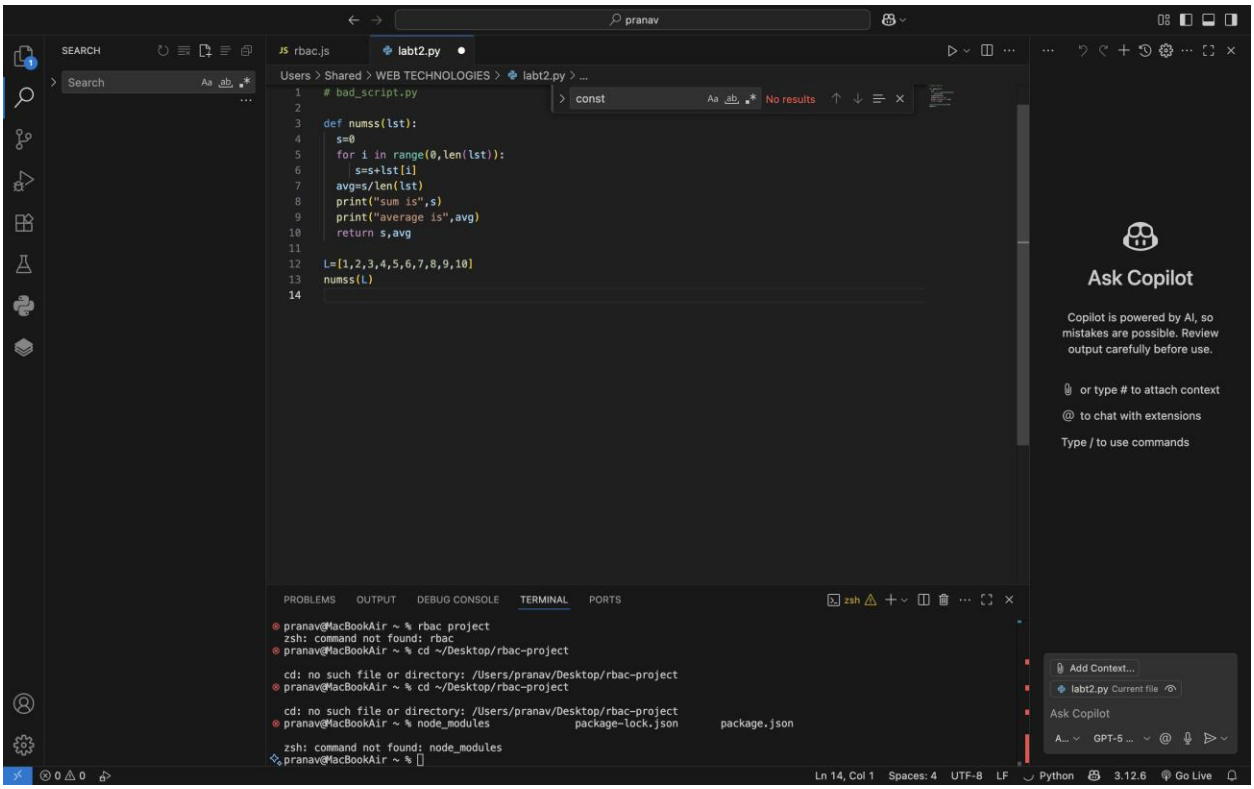
Task 2:

Ask the AI to rewrite the code following PEP 8 guidelines and include explanations of
improvements in naming, indentation, and structure

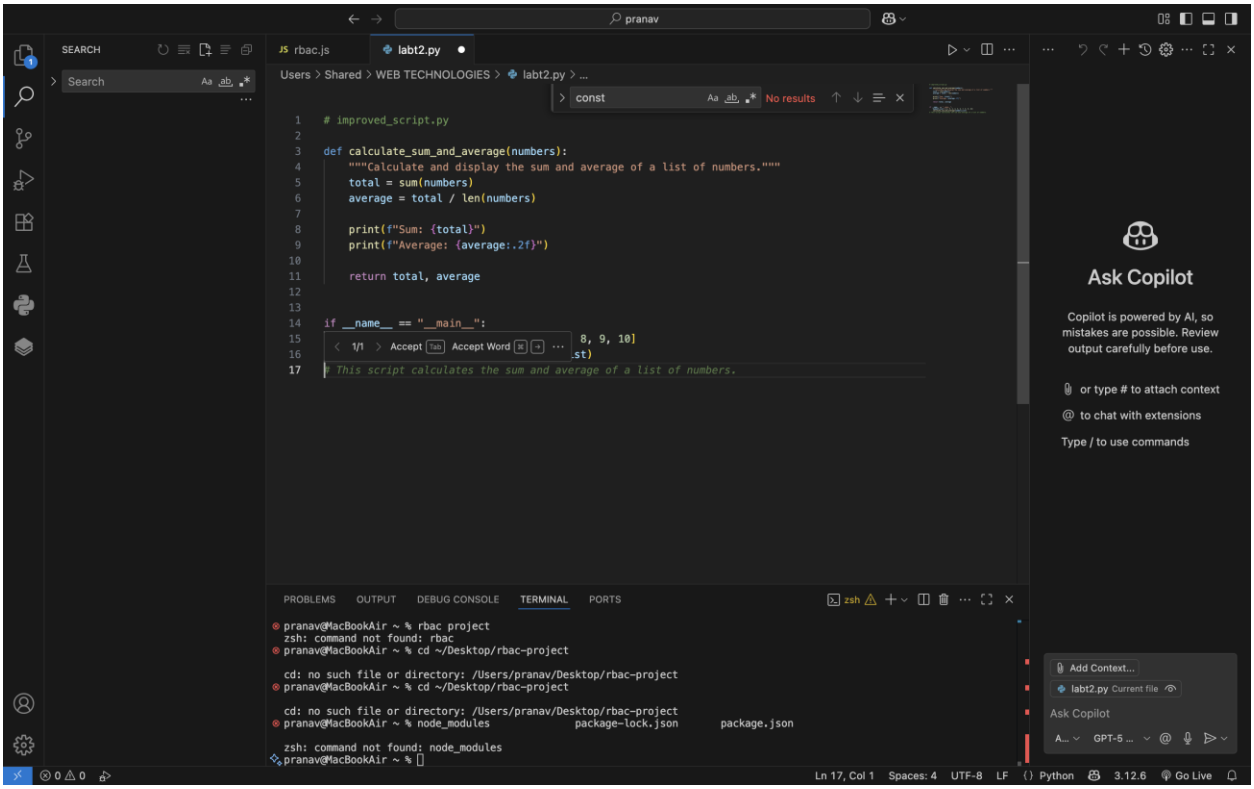
PROMPT :

Rewrite the below Python code following **PEP 8 guidelines**.
Improve **naming, indentation, and structure** for clarity and efficiency.
Explain the **changes made and why they improve the code**.

BAD PYTHON SCRIPT :



IMPROVED PYTHON SCRIPT OR CODE :



OUTPUT :

Function Name.

BEFORE :numss,AFTER : calculate_sum_and_average,

>Clear, descriptive name following snake_case convention.

Variable Names

BEFORE :s, L, i,AFTER : total, numbers_list, numbers,

>Improved readability and meaning.

Loop Logic,

BEFORE : Manual summation with for loop, AFTER :Used built-in sum() function,

> More efficient and Pythonic.

Indentation,

BEFORE : Inconsistent spacing,AFTER: 4 spaces per indent,

>Complies with PEP 8 standard.

Structure,

BEFORE : No entry point, AFTER :Added if __name__ == "__main__":, >Makes code modular and reusable.

Docstring,

BEFORE : None, AFTER :Added descriptive docstring,

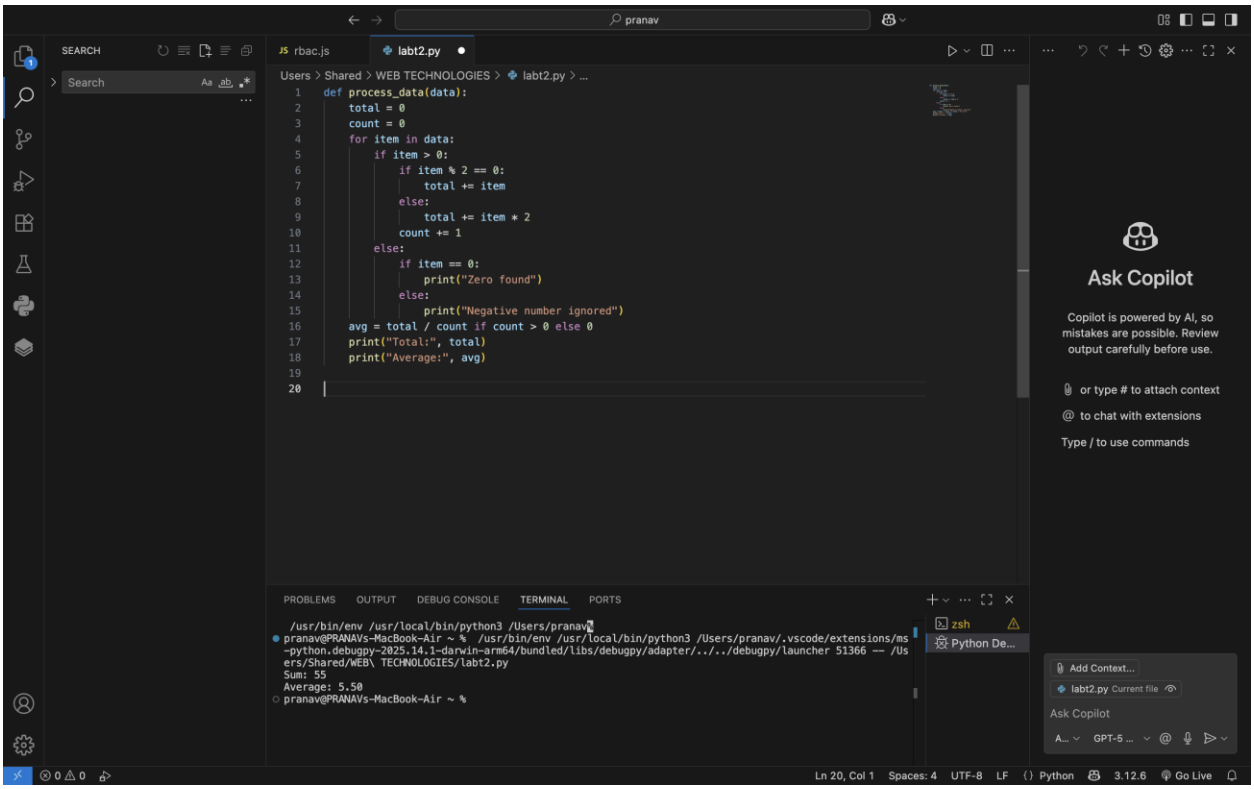
> Explains function purpose clearly.

Question 2: Improving Readability and Maintainability
Task 1:
Provide AI with a long, nested function (e.g., multiple if-else or loops). Ask it to refactor the
code into modular, readable functions.
Task 2:
Use AI to generate code comments and a short design summary, explaining how refactoring improved maintainability and clarity.

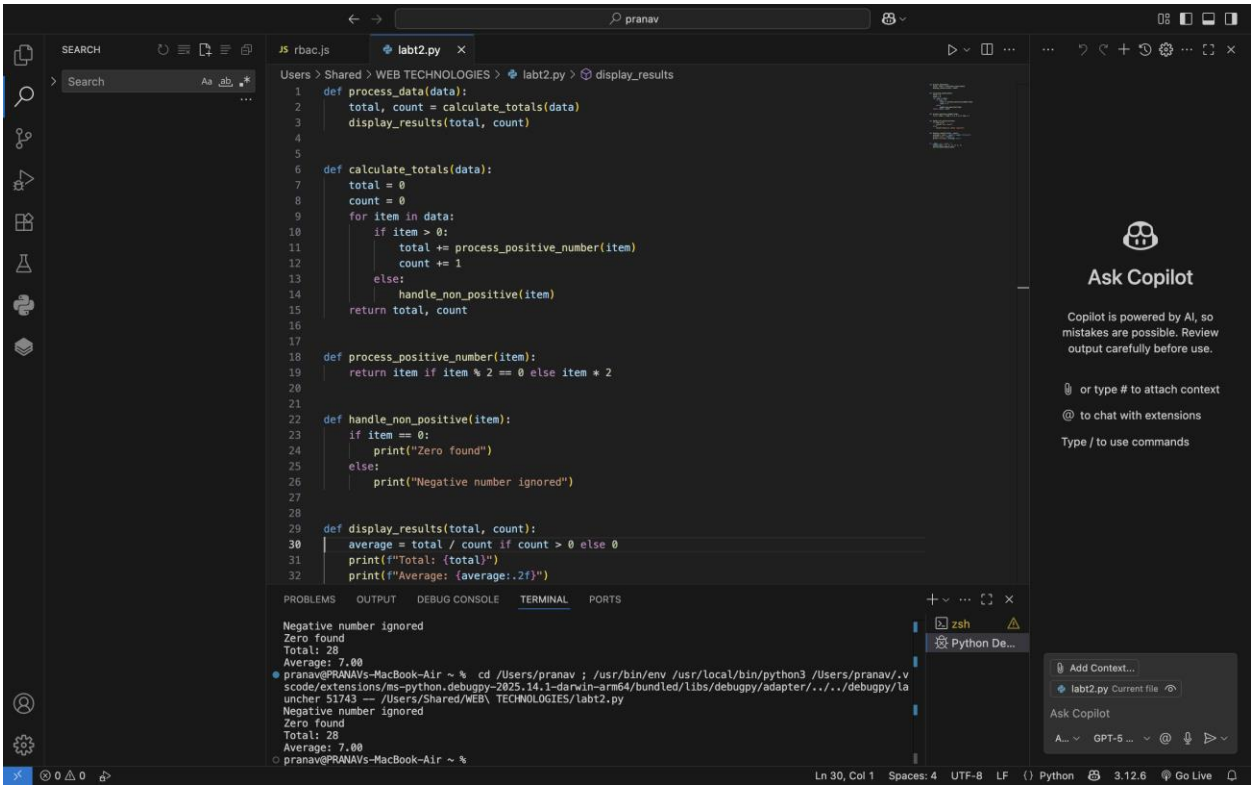
PROMPT :

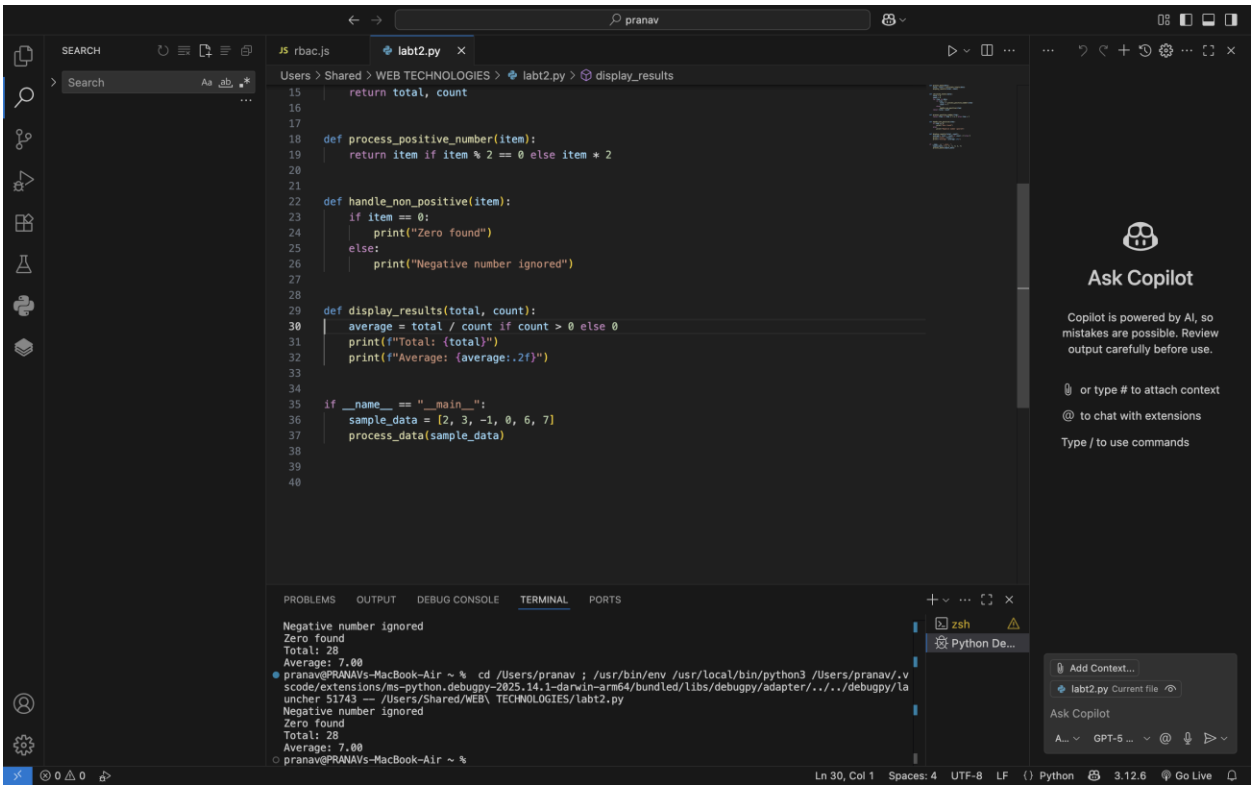
Refactor the following long, nested Python function into **modular and readable functions**.
Make the code easier to understand and maintain.
Add **clear comments** and provide a **short design summary** explaining how refactoring improved readability and maintainability.

INITIAL CODE :

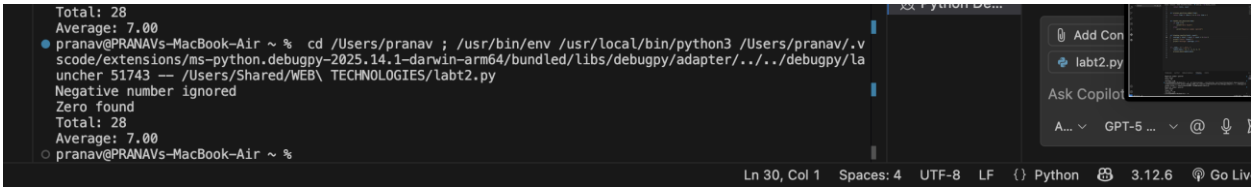


FINALIZED CLEAN CODE :





OUTPUT :



OBSERVATION :

Aspect, Before, After, Explanation

Structure,

Before: Single long, nested function,AFTER :

Split into multiple helper functions,

> Improves readability, allows reuse, and isolates logic.

Naming,

BEFORE : Generic variables and function name, AFTER :

Descriptive names (calculate_totals, process_positive_number),

>Clarifies purpose of each part.

Comments & Docstrings,

BEFORE : None,

AFTER : Added clear docstrings,

>Helps other developers understand purpose of each function.

Complexity, BEFORE :

Deeply nested if-else blocks, AFTER :Moved conditions into smaller functions,
>Reduces cognitive load and makes debugging easier.

Maintainability

BEFORE :Hard to modify or test,

AFTER :

Modular, self-contained functions,

> Easier to update or extend individual parts.